University of Pavia, AA 2025/2026

Computational methods in Physics

Simone Rodini simone.rodini@unipv.it

Abstract

Lecture notes for the Computational methods in Physics class, within the Physical Sciences master degree of University of Pavia.

These notes are brief, and their purpose is to collect the main results and expressions, so that they can be used as a reference for an actual implementation.

With the notes are available a number of Mathematica notebooks, that serves as support material.

Contents

I Approximation and differentiation 3

1 Polynomial approximation 3

2 Nodal and Modal expansions 4
- 2.1 Nodal expansion . . . 4
- 2.2 Modal expansion . . . 4

3 Barycentric forms 6

4 Runge phenomenon 7

5 Chebyshev polynomials 7

6 Chebyshev barycentric weights 10

7 Differentiation on the grid 11

8 Non-linear mappings 14
- 8.1 First example . . . 14
- 8.2 Second example . . . 16
- 8.3 Final results . . . 16

9 Beyond single-grid construction 16
- 9.1 $\mathcal{C}^0$ tessellation . . . 17

II Numerical Quadrature 19

III Integration of Differential Equations 20

Bibliography 21

Part I

Approximation and differentiation

1 Polynomial approximation

In the course, we will only cover the case of mono-dimensional (or univariate) problems. Let us simply mention that the multivariate case, with factorized domains (i.e. squares, cubes, etc) can be trivially obtained by combining multiple univariate constructions. If the domain is non-factorizable, then dedicated methods should be employed.

Suppose that we want to obtain a reliable approximation of a function $f(x)$, which might be very expensive to compute, so that we can generate the approximation once, and then use it instead of the function itself. This will be particularly useful for solving quadrature problems, and certain types of differential equations.

The point is how to construct such an approximation. We will take the general approach of using polynomials to approximate our solution. How to construct a polynomial in practice, we will see in a bit. For the moment, we state the Weierstrass approximation theorem, upon which relies the fact that we can expect a polynomial to perform sufficiently well as an approximant:

Weierstrass approximation theorem: If f is any continuous function on the finite closed interval $[a, b]$, then for every $\varepsilon > 0$ there exists a polynomial $p_n(x)$ of degree $n = n(\varepsilon)$ such that

$$\max_{x\in[a,b]} |f(x) - p_n(x)| < \varepsilon \tag{1}$$

□

There are two, conceptually distinct ways to go about finding a polynomial approximation of a given function. Both approaches rely on the definition of a polynomial basis that will define how the approximant is constructed. We will cover number of different basis, but the most famous are

$$\mathcal{M}_p = \{1, x, x^2, ..., x^p\} \tag{2}$$

which is nothing more than the natural basis for the polynomials of degree $\leq p$;

$$\mathcal{L}_p\left(\{x_i\}_{i=0}^p\right) = \{l_j(x; \{x_i\}_{i=0}^p)\}_{j=0}^p, \qquad l_j(x; \{x_i\}_{i=0}^p) = \prod_{\substack{i=0\\ i\neq j}}^{p} \frac{x - x_i}{x_j - x_k} \tag{3}$$

that is the Lagrange basis for the given set of nodes $\{x_i\}_{i=0}^p$, where note that the nodes are exactly $p+1$, where p is the degree of each of the polynomials $l_j(x)$;

$$\{P_n(x)\}_{n=0}^p \tag{4}$$

which are the Legendre polynomials of degree $\leq p$. Note that the Legendre polynomials are just one example of a special class of polynomial families that are orthogonal with respect to some given inner product. In this case, the inner product looks like

$$\langle P_n | P_m \rangle = \int_{-1}^{1} dx P_n(x) P_m(x) = \frac{2}{2n+1}\delta_{n,m} \tag{5}$$

Let us note that neither the natural basis not the Lagrange basis are, in general, orthogonal basis.

Now, we can define the two different methods to find a polynomial approximation of a function $f(x)$ as nodal expansions versus modal expansions. The former is the easier to understand.

2 Nodal and Modal expansions

2.1 Nodal expansion

Fix a polynomial basis of degree p, with basis elements $b_j(x)$ then we can construct an approximation

$$\phi(x) = \sum_{j=0}^{p} c_j b_j(x) \quad \text{such that} \quad f(x_i) = \phi(x_i) \tag{6}$$

for a given set of exactly $p+1$ distinct nodes x_i. Looking at the requirements, this should immediately point towards the Lagrange polynomials, and indeed they are essentially the polynomial basis of choice for nodal expansions. It should also be apparent why the expansion is named nodal: we are fixing a set of nodes, on which we require that the approximation and the function itself are exactly equal. We can construct a bit of intuition why the Lagrange basis is the most obvious choice in this case. First of all, by the definition itself we see that

$$l_j(x_i) = \delta_{ij} \tag{7}$$

then, we have

$$\phi(x) = \sum_{j=0}^{p} c_j l_j(x) \Rightarrow \phi(x_i) = \sum_{j=0}^{p} c_j l_j(x_i) = \sum_{j=0}^{p} c_j \delta_{ij} = c_i \tag{8}$$

therefore, using the Lagrange basis, we immediately have that $c_i = f(x_i)$, which gives us a very straightforward way to construct the polynomial interpolation.

What would have happened if we were to choose a different basis? For instance, let us take the canonical polynomial basis:

$$\phi(x) = \sum_{j=0}^{p} c_j x^j \Rightarrow \phi(x_i) = \sum_{j=0}^{p} c_j x_i^j \tag{9}$$

Now, we can use a very common 'trick': let us collect $\phi(x_i)$ into a vector: $\boldsymbol{\phi} = \{\phi(x_0), ..., \phi(x_p)\}$, and let us do the same for the coefficients $\boldsymbol{c}$ and for the matrix $B_{ij} = x_i^j$. Then, the system of equations to determine the coefficients becomes

$$\boldsymbol{\phi} = B\boldsymbol{c} \Rightarrow \boldsymbol{c} = B^{-1}\boldsymbol{\phi} \tag{10}$$

Since we choose a basis for our polynomials, and we have the constant assumption that all the nodes are different from each other, then the matrix is guaranteed to be invertible (a zero eigenvalue in the matrix would signify a non-trivial set of nodes for which the linear independence of the basis functions would be violated, hence a contradiction). The problem is that, numerically, taking the inverse is both slow and potentially very inaccurate.

This procedure is not limited to the two set of polynomial basis, but can actually be employed for any polynomial basis, with the only modification being in the matrix B. We can also immediately see that the Lagrange basis is nothing more than the basis that diagonalizes the matrix B directly.

2.2 Modal expansion

The nodal expansion works in a rather general context, since the only required input is the choice of the polynomial basis and the selection of the nodes. The modal expansion instead requires the specification of the polynomial basis and an inner product on the space of polynomials. We saw that the Legendre polynomials are orthogonal with respect to the inner product with the unit function on the interval $[-1, 1]$.

As for the case of the nodal expansion, where the Lagrange polynomials played a special role, so here the entire class of orthogonal polynomials play a special role. Let us write again the polynomial approximation as

$$\phi(x) = \sum_{j=0}^{p} c_j b_j(x) \tag{11}$$

but now we want to derive the expansion coefficients by imposing the equality between the approximant and exact moments (or modes) of the function, i.e.

$$\langle f|b_j\rangle_\omega = \langle \phi|b_j\rangle_\omega \tag{12}$$

where ω defines the inner product as

$$\langle f|g\rangle_\omega = \int_{-1}^{1} dx\omega(x)f(x)g(x) \tag{13}$$

where we assume that the function $\omega(x) \geq 0$ for all $x \in [-1, 1]$. Note that here we will always assume that the domain is $[-1, 1]$. One can move to arbitrary compact domain via a simple linear transformation.

Here it becomes rather evident why orthogonal polynomials are the convenient choice: from

$$\langle b_i|b_j\rangle_\omega = n_i\delta_{i,j} \tag{14}$$

where n_i is some normalization, we immediately have that

$$\langle \phi|b_j\rangle_\omega = n_j c_j \Rightarrow c_j = \frac{1}{n_j}\langle f|b_j\rangle_\omega \tag{15}$$

We note that, typically, it is more computationally heavy determine the coefficient of a modal expansion compared to a nodal one. However, in some context it is very convenient being able to determine the expansion of an arbitrary function onto a basis of polynomials. As an example, consider the following integro-differential problem that emerges in the study of perturbative-QCD

$$\frac{\partial f(x,\mu)}{\partial \log \mu^2} = \int_x^1 dz C(z) f\left(\frac{x}{z}\right) \tag{16}$$

where $f(x) \neq 0$ in $[0, 1]$. Note that this problem cannot really be solved in this form. Then, assume that we are happy with an approximated solution, specifically, we are happy to have a polynomial expansion in the canonical basis. We could try to construct a nodal approximation (which is one of the current methods employed to solve such equations to a high degree of accuracy) or we could try to use a modal expansion. With the latter, we have

$$\begin{aligned}\frac{\partial \langle f(x,\mu)|x^n\rangle}{\partial \log \mu^2} &= \int_0^1 dx x^n \int_x^1 dz C(z) f\left(\frac{x}{z}\right) \\ &= \int_0^1 dz C(z) \int_0^z dx x^n f\left(\frac{x}{z}\right) \\ &= \int_0^1 dz C(z) z^{n+1} \int_0^1 dt t^n f(t)\end{aligned} \tag{17}$$

or, in other words

$$\frac{\partial \gamma_n(\mu)}{\partial \log \mu^2} = C_n \gamma_n(\mu) \tag{18}$$

where $\gamma_n(\mu) \equiv \langle f(x,\mu)|x^n\rangle$ and

$$C_n = \int_0^1 dz C(z) z^{n+1} \tag{19}$$

Now this is trivially solved as

$$\gamma_n(\mu) = \left(\frac{\mu}{\mu_0}\right)^{2C_n} \gamma_n(\mu_0) \tag{20}$$

This is especially powerful since, in practice, one may only be able to compute C_n instead of the full function $C(z)$, because the extra integral, simplifies the computation of the corresponding perturbative diagrams.

3 Barycentric forms

The naïve construction of the Lagrange interpolating polynomial

$$l_j(x) = \prod_{\substack{i=0 \\ i \neq j}}^{p} \frac{x - x_i}{x_j - x_i} \tag{21}$$

is actually not very good in practice, both from a numerical stability point of view and from a performance point of view.

The first improvement that we can make to the formula is to note that we can precompute, once the grid is known, the weights

$$\lambda_j = \prod_{\substack{i=0 \\ i \neq j}}^{p} (x_j - x_i)^{-1} \tag{22}$$

Moreover, we can define a unique polynomial of degree $p+1$ that has zeros on exactly the $p+1$ nodes of the grid

$$l(x) = \prod_{j=0}^{p} (x - x_j) \tag{23}$$

which is sometimes referred to as the node polynomial. Now, we can combine this two constructions to obtain a fairly simple alternative representation for the Lagrange polynomials

$$l_j(x) = \frac{\lambda_j l(x)}{x - x_j} \tag{24}$$

this is called the <u>first barycentric formula</u> of the Lagrange interpolant. What is the computational advantage here?

In the original formulation, to compute the approximant of a function

$$p(x) = \sum_{j=0}^{p} f_j l_j(x) \tag{25}$$

we needed to take $O(p)$ additions, and for each addition we needed $O(p)$ multiplications, therefore the total computational complexity comes out to be $O(p^2)$, which is typically not very good. With the first barycentric formula instead, we still need $O(p^2)$ operations to compute the weights λ_j, but this is a one-time-only computation, therefore it does not really matter that much. Once we have them, we do not have anymore nested sums and products: the product that defines $l(x)$ factorizes from the sum over j:

$$p(x) = l(x) \sum_{j=0}^{p} \frac{\lambda_j f_j}{x - x_j} \tag{26}$$

therefore the total computational complexity comes out to be $O(p + p) \sim O(p)$.

A useful observation is that we can obtain the weights as the derivative of the node polynomial evaluated on the grid nodes. Specifically

$$\lambda_j = \frac{1}{l'(x_j)} \tag{27}$$

The proof is by direct computation, and is left as a simple exercise.

There exist an even simpler formula though. We can notice that the polynomial approximant for the function $g(x) = 1$ is

$$\sum_{j=0}^{p} l_j(x) \tag{28}$$

but since the function itself is a polynomial, and we know that the polynomial interpolant is unique, we have immediately that

$$\sum_{j=0}^{p} l_j(x) = 1 \Rightarrow l(x) \sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} = 1 \Rightarrow l(x) = \left[\sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} \right]^{-1} \tag{29}$$

hence

$$p(x) = \left[\sum_{j=0}^{p} \frac{f_j \lambda_j}{x - x_j} \right] \left[\sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} \right]^{-1} \tag{30}$$

which is called the second barycentric formula of the Lagrange interpolant.

There are proofs about the stability of the two formulas for different choices of the underlying grid. Specifically, for an arbitrary grid the first barycentric formula has stronger stability properties compared to both the naïve formulation and the second barycentric formula. It also behave quite well under extrapolation outside the interpolation domain. However, it suffers from potentially catastrophic finite-precision problems (under- and over-flows due to the finite precision of the representation of numbers on a computer). If one steers away from generic grids and restrict itself to special types of grid that have their points clustered near the edges of the domain (more on in in a moment), then the second barycentric formula works very well inside the interpolation region without suffering from finite-precision problems, whereas it is still not stable for the extrapolation region.

Conceptually, the main advantage of the second barycentric formula is that one can uniformly scale all the weights by some constant amount $\lambda_j \to c\lambda_j \ \forall j$ and the expression is unaffected. This is not true for the first barycentric formula, which has a linear dependence over the weights. It is simple to observe that the weights grow extremely fast (exponentially fast in fact) with the number of nodes in the grid. Therefore, once we have a lot of nodes, the weights are so big that cannot really be represented on a computer.

4 Runge phenomenon

To convince ourselves that using equispaced grids is almost never a good idea, we can try to interpolate the function

$$\frac{1}{1 + 25x^2} \tag{31}$$

on the standard interval $[-1, 1]$. If one does it on a grid of equispaced points, the more points one uses the worse the approximation of the function near the boundary of the intervals!

5 Chebyshev polynomials

One of the central basis of orthogonal polynomials in approximation theory is the basis defined by the Chebyshev polynomials. These can be defined in different ways. The most compact representation, which seems a bit obscure at first glance, is

$$T_n(\cos(\theta)) = \cos(n\theta) \tag{32}$$

where we note that the support of the Chebyshev polynomials is always assumed to be the standard interval $[-1, 1]$. With this representation, it can be verified that they are orthogonal with respect to the scalar product

$$\begin{aligned} \langle T_n | T_m \rangle_{1/\sqrt{1-x^2}} &= \int_{-1}^{1} \frac{dx}{\sqrt{1-x^2}} T_n(x) T_m(x) \\ &= \int_0^{\pi} d\theta \cos(n\theta) \cos(m\theta) = \frac{\pi}{2} \delta_{nm} \left(1 + \delta_{n0}\right) \end{aligned} \tag{33}$$

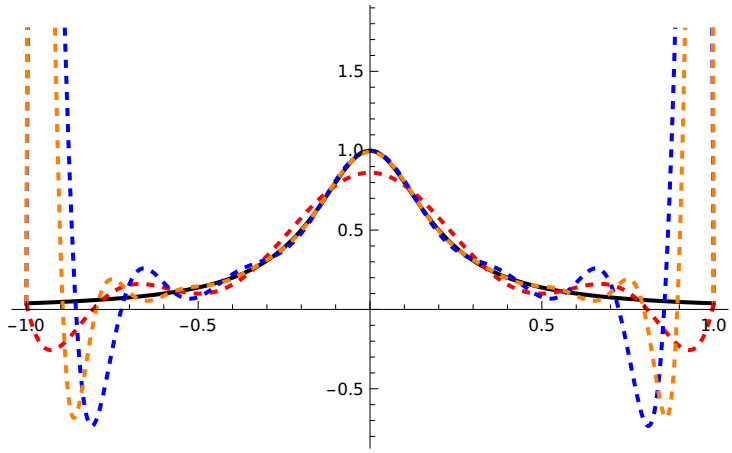

Figure 1: Exemplification of the Runge phenomenon. Solid black line is the true function, then in red, blue and orange respectively the interpolations with 10, 15, and 20 nodes.

Although nice and compact, this representations is anything but clear. How can we be sure that T_n is in fact a polynomial? For once, we can immediately see that

$$T_0(\cos(\theta)) = 1, \quad T_1(\cos(\theta)) = \cos(\theta) \tag{34}$$

i.e.

$$T_0(x) = 1, \quad T_1(x) = x \tag{35}$$

Now, we can state the an inductive construction for the Chebyshev polynomials as

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \tag{36}$$

This can be easily verified to hold true for the cos representation above. And this recurrence relation clearly shows that T_n is a polynomial in its inputs for all n.

The more or less natural question is now: why the cosine? To understand it, assume to have a function f defined from our standard interval to the real numbers that is regular enough. Then, we can always write f as a modal expansion onto the basis of Chebyshev polynomials as

$$f(x) = \sum_{j\geq 0} a_j T_j(x) \tag{37}$$

Now consider a related function, that is defined by $F(\theta) = f(\cos(\theta))$, for which we obviously have that $F(\theta) = F(-\theta)$. For this function we can write the Fourier series as

$$F(\theta) = \sum_{k\in\mathbb{Z}} c_k e^{ik\theta} \tag{38}$$

where we have a series and not an integral because we are working on a compact interval and therefore have only numerably many distinct momentum modes, instead of a continuum spectrum. Since we know that the function is even, we reduce the Fourier series in half, by noticing that the negative momentum modes must be identical to the positive ones, i.e.

$$F(\theta) = \sum_{k\geq 0} c_k \left(e^{ik\theta} + e^{-ik\theta}\right) = \sum_{k\geq 0} 2c_k \cos(k\theta) = \sum_{k\geq 0} 2c_k T_k(\cos(\theta)) \tag{39}$$

But we also have that

$$F(\theta) = f(\cos(\theta)) = \sum_{j\geq 0} a_j T_j(\cos(\theta)) \tag{40}$$

Since the expansion is done on a polynomial basis we automatically have that

$$2c_k = a_k \quad \forall k \tag{41}$$

In essence, the expansion on a Chebyshev basis is nothing more that a discrete cosine transform!

To make the connection complete, we need to specify what we mean by f being 'regular enough'. It is to consider f to be of uniform bound variation on the interval, which is to say that it should be Lipschitz-1, i.e. $\exists C$ such that $|f(x) - f(y)| \leq C|x - y|$ $\forall x, y \in [-1, 1]$. In words, we can always find a finite constant C so that the variation of f on the interval is bounded by the separation between the points times the constant.

Then we can state the theorem
Chebyshev Series if $f : [-1, 1] \to \mathbb{R}$ is Lipschitz-1, then it has a unique Chebyshev series, which converges both absolutely and uniformly. $\square$

Leveraging again the connection between the Fourier representation and the Chebyshev one, we can introduce the Chebyshev-Lobatto points. To do so, fix an integer n, and consider the $n + 1$ equispaced points on the top-half of the unit circle, so that $\theta_j = \pi(j/n)$ for $j = 0, .., n$. The cartesian coordinates of these points are clearly $(\cos(\theta_j), \sin(\theta_j))$. If we take the angles θ_j to be grid nodes for the Fourier representation, then we immediately have that grid nodes for the Chebyshev representation are the cosines of those angles, i.e.

$$t_j = \cos(\theta_j) = \cos\left(\frac{j\pi}{n}\right) \tag{42}$$

It is immediate to see that these are the unique $n + 1$ points on which $T_n(x) = \pm 1$.

Why are we introducing these points? To answer we need to consider two different ways to construct an approximation of a function f using Chebyshev polynomials.

On the one side, we could use directly the modal expansion, truncated to some maximum degree, which is nothing more that the partial sum of the Chebyshev series. Let us define it as

$$f_p(x) = \sum_{j=0}^{p} a_j T_j(x) \tag{43}$$

On the other side, we want to construct a nodal interpolation, on the Chebyshev-Lobatto points. We know that the Lagrange polynomial is the unique interpolant. We also know that the Lagrange polynomials form a basis for the polynomials of a given degree, and so does the Chebyshev basis. Therefore, we can write the interpolant using the Chebyshev polynomials instead of the Lagrange ones as

$$p_n(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{44}$$

Do not be tempted to interpret immediately $c_j = a_j$! c_j are defined by the interpolating construction, i.e.

$$p_n(t_k) = f(t_k) \tag{45}$$

which will give a system of linear equations in the c_j, whereas there is no guarantee that $f_p(t_k) = f(t_k)$!

However, intuition says that some connection must exist. Indeed, we can prove by direct computations that for fixed $p \geq 1$ and for any $m \in [0, p]$ the Chebyshev polynomials $T_{2jp\pm m}$ take the same values as T_m on the $p + 1$ Chebyshev-Lobatto points for all $j > 0$.

With this, we can take the full Chebyshev of f, evaluate it in turn on each Chebyshev-Lobatto point as use the statement above, together with the nice convergence properties of the series, to reorder terms. Specifically, in the full series we re-arrange terms so that we have a tower of combinations of Chebyshev polynomials evaluated on the grid points on which they all take the same values. This defines the coefficients c_k of the interpolant in terms of the coefficients of the full series as

$$c_0 = a_0 + \sum_{j\geq 1} a_{2jp} \tag{46}$$

$$c_k = a_k + \sum_{j\geq 1} a_{k+2jp} + a_{-k+2jp} \tag{47}$$

$$c_p = a_p + \sum_{j\geq 1} a_{p+2jp} \tag{48}$$

This, by construction, ensures that

$$p_n(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{49}$$

interpolates the function on the grid.

This is obviously not a practical recipe to compute the interpolant! It is, however, very important from the theory point of view, because it ensures that the interpolants converge absolutely to the original function, which is a very useful property to have.

6 Chebyshev barycentric weights

Another major advantage of the Chebyshev-Lobatto points, is they allow for a closed-form computation of the interpolation weights, which immensely reduces the number of computations required in practice, not to mention the numerical stability of the procedure.

Let us start with some intermediate results. First, we have a representation for the Lagrange polynomial

$$l(x) = 2^{-p}\left(T_{p+1}(x) - T_{p-1}(x)\right) \tag{50}$$

To prove it, we know that $l(x)$ is defined by its $p+1$ roots, that for us are the $p+1$ Chebyshev-Lobatto points t_j. Since the polynomial on the right-hand side is also of degree $p+1$, if we can show that has the exact same roots, we have established the proportionality. Then, since the coefficient of x^{p+1} of $l(x)$ is exactly 1, we can just show that the same is true for the normalized right-hand side, and we are done.

First, let us compute

$$T_{p+1}(t_j) = \cos\left(\frac{(p+1)j\pi}{p}\right) = \cos(j\pi)\cos\left(\frac{j\pi}{p}\right) - \sin(j\pi)\sin\left(\frac{j\pi}{p}\right) \tag{51}$$

$$T_{p-1}(t_j) = \cos\left(\frac{(p-1)j\pi}{p}\right) = \cos(j\pi)\cos\left(\frac{j\pi}{p}\right) + \sin(j\pi)\sin\left(\frac{j\pi}{p}\right) \tag{52}$$

$$\tag{53}$$

Since $\sin(j\pi) = 0$, we immediately have that $T_{p+1}(t_j) - T_{p-1}(t_j) = 0$ for all $j \in [0, p]$. Now, from the defining recurrence relation of the Chebyshev polynomials we have that

$$T_n(x) = 2^{n-1}x^n + ... \tag{54}$$

if $n > 0$, from which immediately follows that

$$T_{p+1}(x) - T_{p-1}(x) = 2^p x^{p+1} + ... \tag{55}$$

from which the equality is established.

Now we can compute the weights. Recall that they can be defined from the derivatives of the Lagrange polynomial at the grid nodes, i.e.

$$\beta_j = \frac{1}{l'(t_j)} = \frac{2^p}{T'_{p+1}(t_j) - T'_{p-1}(t_j)} \tag{56}$$

where we switched to a dedicated notation for the weights, to remember that these are for the Chebyshev points only.

Now, we can compute the derivatives of the Chebyshev polynomials, using a very simple trick:

$$\frac{d}{dx}T_n(x) = \frac{d}{d\cos\theta}T_n(\cos\theta) = -\frac{1}{\sin\theta}\frac{d}{d\theta}T_n(\cos\theta) \Rightarrow \tag{57}$$

$$
\begin{aligned}
T'_{p\pm1}(t_j) &= -\frac{1}{\sin\theta_j}\frac{d}{d\theta_j}\cos\left((p\pm1)\theta_j\right) \\
&= -\frac{1}{\sin\theta_j}\frac{n}{\pi}\frac{d}{dj}\cos\left(j\pi\pm\frac{j\pi}{p}\right) \\
&= \frac{1}{\sin\theta_j}\frac{n}{\pi}\left(\pi\pm\frac{\pi}{p}\right)\left[\sin(j\pi)\cos\left(\frac{j\pi}{p}\right)\pm\cos(j\pi)\sin\left(\frac{j\pi}{p}\right)\right]
\end{aligned}
\tag{58}
$$

Now, if $j \neq 0, p$ we can trivially evaluate the expression, and get

$$
T'_{p\pm1}(t_j) = \pm(-1)^j(p\pm1) \tag{59}
$$

if $j = 0, p$, we need to be a bit more careful because the first one over sine diverges. No matter, taking the appropriate limit (by symmetry we have that the results for $j = p$ must be equal, up to a global $(-1)^p$ to the results of $j = 0$):

$$
T'_{p\pm1}(t_0) = (p\pm1)^2 \tag{60}
$$

Therefore, we have that

$$
\beta_0 = \frac{2^{p-1}}{p}\frac{1}{2} \tag{61}
$$

$$
\beta_{j\neq0,p} = \frac{2^{p-1}}{p}(-1)^j \tag{62}
$$

$$
\beta_p = \frac{2^{p-1}}{p}\frac{(-1)^p}{2} \tag{63}
$$

It is immediate to see that we can define a scaled down version of the weights by

$$
\tilde{\beta}_j = \frac{p}{2^{p-1}}\beta_j \tag{64}
$$

It is also convenient for the following discussions to extract the phase from the weights, and write

$$
\gamma_j = \tilde{\beta}_j(-1)^j \tag{65}
$$

having essentially that $\gamma_{0,p} = 1/2$ and $\gamma_{j\neq0,p} = 1$.

Therefore, our basis functions for the Chebyshev interpolant on the Chebyshev-Lobatto points is greatly simplified, especially in the second barycentric representation:

$$
p(x) = \sum_{j=0}^{p} f(t_j)b_j(x), \quad b_j(x) = \frac{\tilde{\beta}_j}{x-t_j}\Bigg/\sum_{i=0}^{p}\frac{\tilde{\beta}_j}{x-t_j} \tag{66}
$$

Now we can establish a connection between the coefficients $f(t_j)$ of the interpolant written in the natural Lagrange basis, and the coefficients c_k for the interpolant written in the Chebyshev basis. This will not be immediately useful, but we will need it when we will talk about quadrature rules. We can write

$$
p(x) = \sum_{j=0}^{p} f(t_j)b_j(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{67}
$$

with

$$
c_j = \frac{2\gamma_j}{p}\sum_{i=0}^{p}\gamma_i T_j(t_i) f_i \tag{68}
$$

7 Differentiation on the grid

Now that we discussed the different ways to construct a polynomial interpolant, we can apply this construction to obtain a method to compute the derivatives of our function in an approximated way.

First, let us derive the results for the generic Lagrange interpolant, and then we will derive specialized representations for the Chebyshev grids.

First the trivial expression

$$p(x) = \sum_{j=0}^{p} f_j l_j(x) \Rightarrow p'(x) = \sum_{j=0}^{p} f_j l_j'(x) \tag{69}$$

but then $p'(x)$, as a function, can be itself interpolated on the grid, to give the representation

$$p'(x) = \sum_{j=0}^{p} p'(x_j) l_j(x) \tag{70}$$

We can use now the first representation into the second, to obtain

$$p'(x) = \sum_{j=0}^{p} l_j(x) \sum_{i=0}^{p} f_i l_i'(x_j) \tag{71}$$

Define the derivative matrix

$$D_{ij} = l_i'(x_j) \tag{72}$$

and we can write

$$f'(x) \sim p'(x) = \sum_{i=0}^{p} f_i \sum_{j=0}^{p} D_{ij} l_j(x) \tag{73}$$

i.e. we have identified

$$l_i'(x) = \sum_{j=0}^{p} D_{ij} l_j(x) \tag{74}$$

which also gives us a formula for repeated differentiation: with the convention that repeated indexes are summed over

$$\frac{d^n}{dx^n} l_{i_0}(x) = D_{i_0 i_1} D_{i_1 i_2} .. D_{i_{n-1} i_n} l_{i_n}(x) \tag{75}$$

Now we can explicitly compute the derivative matrix. First, we note that we have

$$\begin{aligned} l_j'(x) &= \frac{d}{dx} \prod_{\substack{k=0 \\ k \neq j}}^{p} \frac{x - x_k}{x_j - x_k} \\ &= \sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \prod_{\substack{k=0 \\ k \neq j,n}}^{p} \frac{x - x_k}{x_j - x_k} \\ &= \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x - x_n} \right] l_j(x) \end{aligned} \tag{76}$$

Now we can use this expression to evaluate it at the grid points. First, we have the diagonal terms

$$l_j'(x_j) = \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \right] l_j(x_j) = \sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \tag{77}$$

For the off-diagonal terms we instead obtain

$$l_j'(x_i) = \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \right] \prod_{\substack{k=0 \\ k \neq j,n}}^{p} \frac{x_i - x_k}{x_j - x_k} = \frac{1}{x_j - x_i} \prod_{\substack{k=0 \\ k \neq j,i}}^{p} \frac{x_i - x_k}{x_j - x_k} \tag{78}$$

where the last equality holds because if $\forall n \neq i$ in the sum, the product contains a single term in the numerator that is $x_i - x_i$, hence it removes the corresponding contribution.

Notice that we obtained the results in the naïve representation for the Lagrange interpolant. It is actually easy to see that an equivalent presentation of the final result is

$$f'(x) \sim p'(x) = \sum_{j=0}^{p} l_j(x) \sum_{i=0}^{p} f_i D_{ij} \tag{79}$$

with this we can define the coefficients $\tilde{f}_j = \sum_{i=0}^{p} f_i D_{ij}$, and with these apply the first or second barycentric formulas. For instance

$$p'(x) = l(x) \sum_{j=0}^{p} \frac{\tilde{f}_j \lambda_j}{x - x_j} \tag{80}$$

and equivalently for the second barycentric formula.

For the Chebyshev-Lobatto grids, we can derive a closed form representation for the derivative matrix. We already saw the derivative of the Chebyshev polynomials in the previous section, but here can can make use of a nice closed form representation for it, which is more convenient for this derivation:

$$T_k'(x) = kU_{k-1}(x) \tag{81}$$

where U_k are the Chebyshev polynomials of the second kind, which can defined via

$$U_k(\cos(\theta)) = \frac{\sin((k+1)\theta)}{\sin(\theta)} \tag{82}$$

They also from a set of orthogonal polynomials, with respect to the metric $\sqrt{1-x^2}$, i.e.

$$\int_{-1}^{1} dx \sqrt{1-x^2}\; U_n(x) U_m(x) \propto \delta_{nm} \tag{83}$$

To derive now the derivative matrix we can rely on the Chebyshev interpolant written as at the end of the last section:

$$p(x) = \sum_{j=0}^{p} c_j T_j(x), \qquad c_j = \frac{2\gamma_j}{p} \sum_{i=0}^{p} \gamma_i T_i(t_j) f_j \tag{84}$$

from which we have

$$p'(x) = \sum_{j=0}^{p} c_j T_j'(x) = \sum_{j=0}^{p} \left[\frac{2\gamma_j}{p} \sum_{i=0}^{p} \gamma_i T_j(t_i) f_i \right] j U_{j-1}(x) \tag{85}$$

Now we know from the generic discussion that if we evaluate $p'(x)$ on the grid node x_j and we look at the coefficient of the i-th value f_i this gives us the entry D_{ij}. So, rearranging a bit the expression above, we reach the following

$$p'(t_j) = \sum_{i=0}^{p} f_i \left[\sum_{k=0}^{p} \frac{2\gamma_k}{p} k T_k(t_i) U_{k-1}(t_j) \gamma_i \right] \tag{86}$$

To take the sums, one can write all the sine and cosine functions in exponential form. A bit of care is needed for the diagonal terms, and for when either of the i, j indexes is $0, p$. No matter, with patience and the help of Mathematica, one can arrive at the following representation

$$D_{00} = -D_{pp} = \frac{2p^2+1}{6} \tag{87}$$

$$D_{ii} = -\frac{t_i}{2(1-t_i^2)} \tag{88}$$

$$D_{ij} = -\frac{\gamma_i}{\gamma_j} \frac{(-1)^{i+j}}{t_i - t_j} \tag{89}$$

This part has been adapted from the discussion given in [1], with the notable exception that the derivative matrix presented here is the transpose of the derivative matrix of the paper, and notation is slightly different.

8 Non-linear mappings

One of the most useful tools when one wants to perform interpolation out in the real world is the ability of re-mapping the support of the desired variable to a standardized support (which we will assume to be $[-1, 1]$ for definiteness, but it could be different as well). If we are constrained to a linear mapping, this would not be so helpful, beside granting us some code re-usability. Instead, it is the usage of non-linear mapping that really elevates our ability to describe functions for which we know the 'important regions'.

We can produce two, real-world examples where a non-linear mapping is important.

First, the parton distribution functions (which describe the distribution of longitudinal momentum of the constituents of the proton). A general simplified formula could be

$$xf(x) \sim x^{-1/3}(1-x)^5 \tag{90}$$

which is a very simplified form of some of the test Ansätze of [1]. In this context, the domain is $x \in (0, 1]$.

Second, consider the deposited dose by a proton beam in water. A simple parametrization as function of the depth could be

$$\frac{\mathcal{D}(z)}{\mathcal{D}_{\text{ref}}} = \frac{1}{1+\exp\left(\frac{z-R}{\omega}\right)}\left[A + \frac{\exp\left(-\lambda/(R-z+\varepsilon)\right)}{(R-z+\varepsilon)^p}\right] \tag{91}$$

where the domain is $z \in [0, R]$ Where a pseudo-realistic choice for the parameters may be

$$A = \frac{1}{4}, \quad p = \frac{3}{2}, \quad \lambda = \frac{1}{4}, \quad \omega = \frac{1}{2}, \quad R = \frac{45}{2}, \quad \varepsilon = 10^{-4} \tag{92}$$

In Fig.4 they are shown the two examples above, which clearly exhibit the sharp feature that were promised.

In Fig. 3 the same plots, with in red the points corresponding to a 30 nodes for both the Chebyshev-Lobatto grid and an equispaced one.

It is quite clear that with these ranges, equispaced grids cannot realistically capture both the sharp behaviors and the more flat and boring region at the same time. Chebyshev-Lobatto points work better, but again they struggle to correctly follow, in the density of points, the feature of the function.

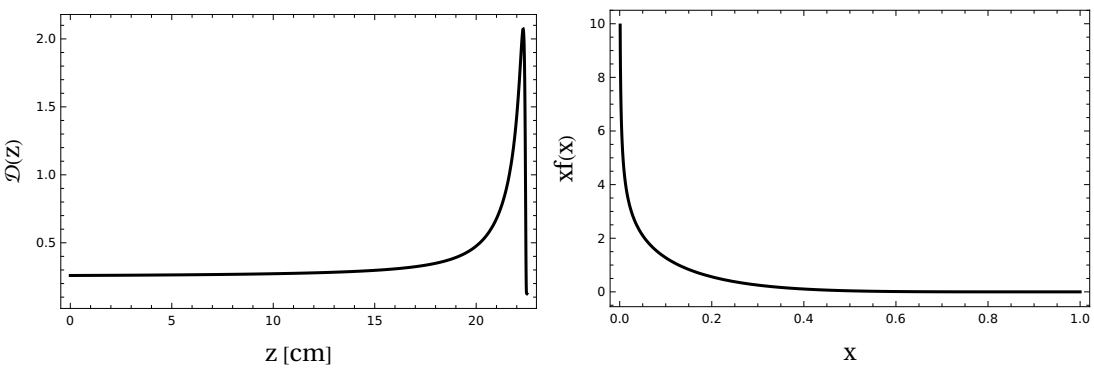

Figure 2: Examples of functions that benefit from non-linear mapping.

8.1 First example

Let us now detail the construction of what is, essentially, the 'industry-standard' mapping that is used for parton distribution functions.

Since the 'sharp feature' in this case is always power-like, and this we can derive from theory, the best mapping is a logarithmic one.

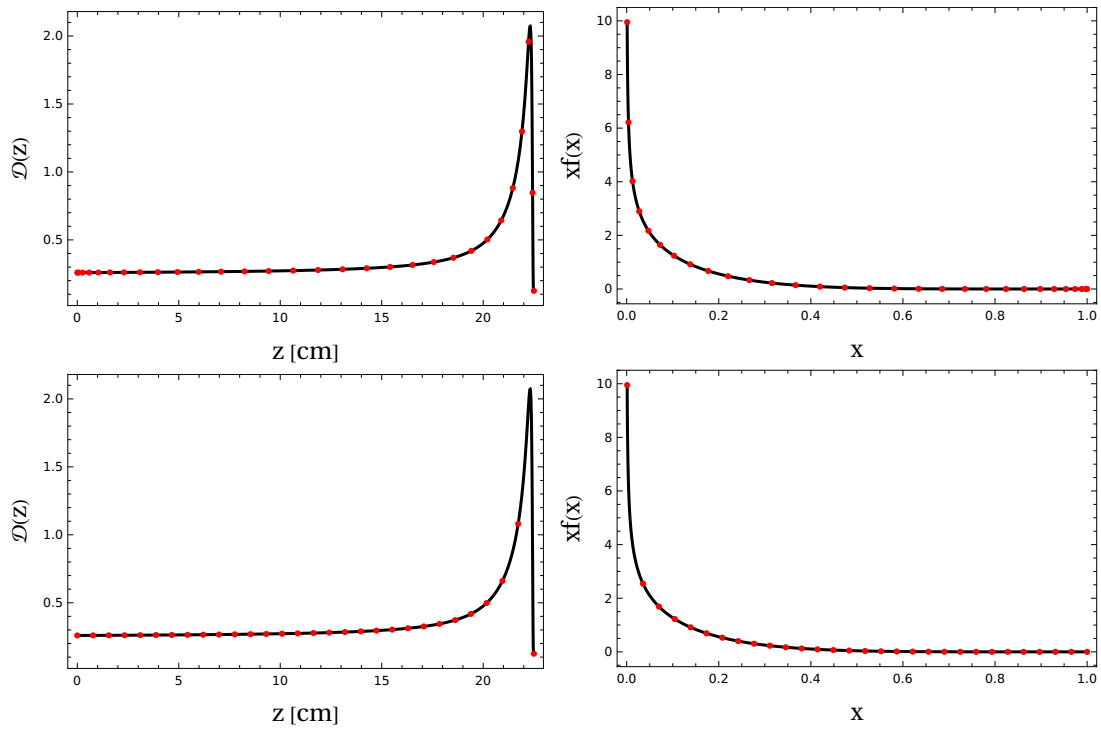

Figure 3: Comparison plot, where in black are the 'exact' functions and in red the points for a grid of 30 nodes. The top panels are with Chebyshev-Lobatto points, the lower ones with equispaced points.

We want to define an intermediate variable u such that

$$u(x) = \log(x) \tag{93}$$

(where log is always the natural log). Since we cannot go to $x = 0$, neither in the original function nor with the variable transformation on top, it is also standard to take the support $x \in [x_0, 1]$, where x_0 is typically small, say 10^{-3} for definiteness (which is what has been used in the plots so far as the lower bound). Now, the support of u is $[\log(x_0), 0]$. Note that, crucially, the function is injective, and that is also monotonically increasing (the first is a requirement, the second just a nice convenient feature). Then, we can introduce the map from u to our normalized space as

$$\tau = 2\frac{u - \log(x_0)}{-\log(x_0)} - 1 \tag{94}$$

One could combine the two, to get a map $x \leftrightarrow t$, but this is not very convenient, since the second linear mapping is actually universal, we just need to know the boundary of u, so, thinking about implementation, we can re-use the linear map for all problems, whereas the original map is problem-dependent.

The two inverse transformations are, trivially

$$u = \frac{\tau + 1}{2}(-\log(x_0)) + \log(x_0) \quad x = \exp(u) \tag{95}$$

Now, we can put a grid in the τ-space: $\{\tau_j\}$ (whether is a Chebyshev-Lobatto grid or something else, doesn't really matter at this stage). This will induce a grid in u-space:

$$u_j = \frac{\tau_j + 1}{2}(-\log(x_0)) + \log(x_0) \tag{96}$$

which in turns induces a grid in x-space, which was our goal

$$x_j = \exp(u_j) \tag{97}$$

8.2 Second example

For the second example, we want more points near the far right end of the support. a reversed log map would work, but for the sake of presenting different maps, let us choose a power-like map, of the form

$$u(z) = z^{\alpha}, \quad z = u^{1/\alpha} \tag{98}$$

Then, the mapping to the normalized space is the same as before, even though now the support of u is $[0, R^{\alpha}]$.

Again the strategy is to put a grid in τ-space, to induce a grid in u-space and, in turn, a grid in z-space.

8.3 Final results

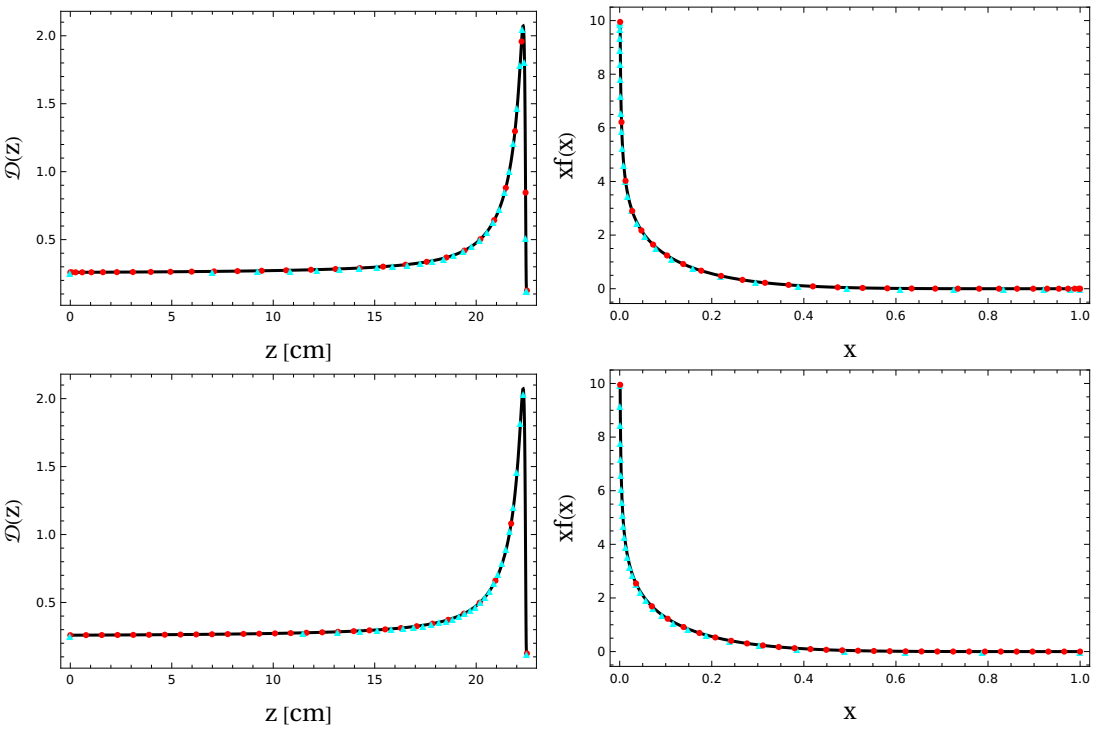

Figure 4: Effects of non-linear mapping. In red circles are the Chebyshev-Lobatto linearly mapped points, whereas in cyan triangles the Chebyshev-Lobatto points with additional non-linear mapping. The bottom panels are the same, but for equispaced grid.

9 Beyond single-grid construction

We saw that, with proper mapping, one can achieve much better densities of the grids where the function has 'important' features. However, we are still limited in what can be achieved in this fashion. One of the most worrying problems is that interpolations with too large polynomial degrees tend to not perform very well (even though some choices are better than others). In practice, constructing polynomial interpolants with degree ≥ 50 is not recommended.

Therefore, we need to find a different approach.

The most versatile alternative, is to construct the global interpolant by a 'collage' of many 'local' ones, under some regularity conditions for the merging points.

We will present to distinct approaches, one that works well for grids like Chebyshev and similar, and the other that relies on the usage of generic grids.

9.1 $\mathcal{C}^0$ tessellation

The simplest, and probably most intuitive, construction is to literally glue together multiple grids, pairwise at the merging points. For instance, say that we want to have a grid that covers $[a, b]$, then we can split the support into $K + 1$ pieces like

$$[a, b] \to [a \equiv x_0, x_1) \cup [x_1, x_2) \cup .. \cup [x_K, b \equiv x_{K+1}] \tag{99}$$

Now, on each interval we can impose a grid (grid_i for the i-th interval), where we assume that the grid_i contains both its extremes points.

To fix the ideas, we can combine in a single map the the non-linear mapping of the previous section and the linear transformation to normalized space. Notice that now each individual grid will have its own linear transformation, since boundaries will be different. We can call the outside variable x, and the inner, standardized variable t. Then, on a single grid, the polynomial interpolant we know we can write as

$$q_i(x) = \sum_{j=0}^{p_i} f(\eta_i(t_j)) b_j(\eta_i^{-1}(x)) \tag{100}$$

where i labels the grid! Since we are requiring that the grids contains both their extreme points, we have a condition on the maps, namely

$$\eta_{i+1}(-1) = \eta_i(1) \tag{101}$$

where we assumed that the normalized space is $[-1, 1]$, in increasing order (this is not the case for the standard Chebyshev-Lobatto grid, for which the points are ordered from $+1$ to -1, but all of what follows applies identically, *mutatis mutandis*)

Now, we can construct the <u>global</u> interpolant as

$$q(x) = \sum_{i=0}^{K} \theta(x \in \text{grid}_i) \sum_{j=0}^{p_i} f(\eta_i(t_j)) b_j(\eta_i^{-1}(x)) \tag{102}$$

where the theta function obviously implements the condition

$$\theta(x \in \text{grid}_i) = \begin{cases} x \in [x_i, x_{i+1}) & \text{if } i \neq K \\ x \in [x_i, x_{i+1}] & \text{if } i = K \end{cases} \tag{103}$$

From now on, to avoid confusion, we will indicate the individual extremes of the intervals via the maps, i.e.

$$\theta(x \in \text{grid}_i) = \begin{cases} x \in [\eta_i(-1), \eta_i(+1)) & \text{if } i \neq K \\ x \in [\eta_i(-1), \eta_i(+1)] & \text{if } i = K \end{cases} \tag{104}$$

Note that we are not requiring that all the grids have the same number of points, in fact we are keeping the freedom of choosing different values of p_i for each i.

Now, Eq. (102) is perfectly fine for interpolation purposes, but often we need to know the individual set of weights associated to each <u>unique</u> grid point. The only common points between adjacent grids are the extreme points, by virtue of Eq. (101). It is therefore instructive to first construct a global grid, that contains only the unique points. Explicitly, we have

$$\left[\bigcup_{i=0}^{K-1} \{\eta_i(t_0), .., \eta_i(t_{p_i-1})\} \right] \cup \{\eta_K(t_0), .., \eta_K(t_{p_K})\} \equiv \{x_0, .., x_{N-1}\} \tag{105}$$

where the total number of unique points N is given by

$$N = 1 + \sum_{i=0}^{K} p_i = -K + \sum_{i=0}^{K} N_i \tag{106}$$

where $N_i = p_i + 1$ is the number of points in the i-th grid, and the last expression makes it clear that we are removing the exactly K common points.

Therefore, our vector of unique function values can be computed on the points x_i. So, we aim to write

$$q(x) = \sum_{l=0}^{N-1} f(x_l) w_l(x) \tag{107}$$

With a little bit of simple algebra, we reach the following representation for the <u>global</u> weights w_l:

$$w_l(x) = \theta(x \in \text{grid}_{i(l)}) b_{j(l)} \left(\eta_{i(l)}^{-1}(x)\right) + \delta_{i(l)\neq 0}\delta_{j(l)=0}\theta(x \in \text{grid}_{i(l)-1}) b_{p_{i(l)-1}-1} \left(\eta_{i(l)-1}^{-1}(x)\right) \tag{108}$$

Horribly verbose... The meaning is although quite clear: for any given index l we know how to compute the underlying indexes $i(l), j(l)$, where the first labels the grid and the second the point on the grid, because we constructed the index l starting from i, j in the first place. Then, we have the first contribution, that is just the natural term in the interpolation formula. If however we are looking at the first point of a subgrid ($\delta_{j(l)=0}$), but avoiding the case of the very first subgrid ($\delta_{i(l)\neq 0}$) which has no subgrid on its left, then we need to include the interpolating polynomial that corresponds to it, since we are storing intervals that are closed on the left and open on the right (if we were to do the opposite, we would need to check if we are sitting at the right-most point of a subgrid).

This final representation also makes it evident that, by construction, the global interpolating function is continuous at the boundary of subgrids. It cannot, however, guarantee any differentiability requirement, because the interpolant from the left and the right of a grid-joining point can reach the point with different slope. This might seem bad, but it is actually a very desirable feature: it means that with this construction we can interpolate functions with discontinuous first derivative at few (known) points. If the function has a discontinuity in the first derivative, and we would try to interpolate it with a smooth polynomials, we would loose a lot of precision.

Conceptually, the construction above does what we want. However, thinking about implementation, this is not ideal. From a code point of view, when we construct and store our joint grid, we can be a little bit smarter.

The global weights are clearly composed of two separate contributions. Therefore, we can define the weights as the individual b_j for each grid in a flatten vector:

$$\tilde{w}_n(x) = \theta(x \in \text{grid}_{i(n)}) b_{j(n)} \left(\eta_{i(n)}^{-1}(x)\right) \tag{109}$$

where the theta function does not need to be included, but can be stored separately since it statically encode the support of the given weight. Now the index n runs over all the points of the grid, including duplicates! We have, however, a non-trivial mapping between the index n and the index l, and this mapping is required to know how to associate each value $f(x_l)$ to its set of weights, and viceversa.

By construction, it should be clear that the map $n \to l$ is 1-to-1, i.e. for a given n, there is only one associated l. The contrary is not true, i.e. $l \to n$ is, in general, 1-to-many. In practice, $l \to n$ is almost always 1-to-1, with the exceptions of the indexes l that satisfy the deltas $\delta_{i(l)\neq 0}\delta_{j(l)=0}$, in this case the map is 1-to-2, i.e. the given l has two associated n values.

It is actually much harder to describe the algorithm in words than to give it to you explicitly, therefore here it is:

```
c_size ← N     compute the total number of unique points
n ← 0
l ← 0
for a ← 0 to K do
    Save current index as the delimiter of two subgrid
    _delim_indexes[a] ← n
    Loop over all points of the a-th grid
    for j ← 0 to grid_deg[a] do
        if j ≠ grid_deg[a] or a = K then
            Compute coordinate in intermediate u space by linear map
            _coord_inter[l] ← lin_map(tj, lims[a].first, lims[a].second)
```

```
            Compute the coordinate in physical x space
            _coord[l] ← to_phys_space(_coord_inter[l])
        end if
        Fill map for n → l
        _from_iw_to_ic[n] ← l
        Fill map for n → i(n), i.e. from index to interval index
        _from_idx_to_inter[n] ← a
        If we are not on the right-most point of an interval, advance l index
        if j ≠ grid_deg[a] then
            l ← l + 1
        end if

        Construct the weights at index n...

        Advance index n
        n ← n + 1
    end for
end for
Append last point to delimiter vector
_delim_indexes[K + 1] ← n
Resize _from_ic_to_iw to size c_size
Loop over all entries in the map n → l
for n ← 0 to size(_from_iw_to_ic) − 1 do
    get l that corresponds to specific n
    l ← _from_iw_to_ic[n]
    Append n to _from_ic_to_iw[l]
end for
```

In this algorithm, the map from n to l `_from_iw_to_ic` is constructed as a plain list of values, one for each n.

The inverse map `_from_ic_to_iw`, from l to n, is constructed as a list of lists, one list for each l that contains all the corresponding n values.

The weights themselves are a list (of functions) indexed by n, whereas the coordinates in u and x space (`_coord_inter` and `_coord` respectively) are a list (of values) indexed by l.

In the pseudo-code above, I am omitting some logic to get the correct values t_j for the a-th sub-grid that is being considered. This is a detail (albeit a practically important one), that it will be clarified in the coding sessions.

Part II

Numerical Quadrature

Part III

Integration of Differential Equations

Bibliography

[1] Markus Diehl, Riccardo Nagar, and Frank J. Tackmann. "ChiliPDF: Chebyshev interpolation for parton distributions". In: Eur. Phys. J. C 82.3 (2022), p. 257. DOI: 10.1140/epjc/s10052-022-10223-1. arXiv: 2112.09703 [hep-ph].